



IPB University
— Bogor Indonesia —

Sekolah
Vokasi

Modul Praktikum

Algoritma dan Struktur Data (TPL2106)

Oleh:

Dr. Sofiyanti Indriasari, S.Kom., M.Kom.

**Teknologi Rekayasa Perangkat Lunak
Semester Genap
Tahun Akademik 2025/2026**



Modul 1

Konsep ADT dan File Handling (Python File I/O)

1. Deskripsi Singkat Modul

Modul Praktikum 1 membekali mahasiswa dengan pemahaman awal tentang abstraksi data (ADT) dan keterampilan dasar pengolahan data berbasis file menggunakan Python, terutama proses membaca/menulis file teks (File I/O) dan memindahkan data dari file ke struktur data di memori (misalnya list/dictionary) sebagai fondasi praktikum berikutnya dan proyek berbasis file.

2. Capaian Pembelajaran

Pada akhir Pertemuan 1, mahasiswa diharapkan mampu:

1. Menunjukkan pengertian algoritma, paradigma pemrograman, dan pengolahan berbasis file.
2. Mengintegrasikan manipulasi file teks sederhana dalam pengolahan data.
3. Menginisialisasi repositori proyek dan membangun modul pembacaan data berbasis file.

3. Prasyarat Pengetahuan Awal

Sebelum mengikuti Praktikum 1, mahasiswa diharapkan telah memiliki kemampuan dasar berikut:

1. Memahami konsep dasar pemrograman: variabel, tipe data, operator, percabangan (if), dan perulangan (for/while).
2. Mampu menjalankan program Python sederhana dan membaca output pada terminal.
3. Memahami struktur dasar program Python (urutan instruksi, blok indentasi).



4. Memahami konsep dasar input-output sederhana (`print()` dan `input()`) sebagai bekal latihan.

4. Kebutuhan Alat

A. Perangkat Keras

Laptop/PC dengan spesifikasi minimum:

- RAM minimal 4 GB (disarankan 8 GB)
- Ruang kosong minimal 1 GB

Koneksi internet (opsional) untuk instalasi awal dan update extension.

B. Perangkat Lunak

- Python 3.x (disarankan Python 3.10+)
- Visual Studio Code (VS Code)
- Extension VS Code: Python (Microsoft)
- Windows Command Prompt / PowerShell / Git Bash

C. File/Folder Praktikum

- Folder kerja praktikum (contoh: Praktikum ASD 1/)
- File dataset teks sederhana (.txt) yang akan digunakan untuk latihan File I/O.

5. Intruksi Ketentuan Teknis Praktikum

- a) Mahasiswa wajib menyimpan seluruh file praktikum dalam satu folder project agar mudah dievaluasi.
- b) Penamaan file mengikuti instruksi modul (misalnya `praktikum1.py`, `data_mahasiswa.txt`).
- c) Setiap kode program harus diberi **komentar** untuk menjelaskan tujuan blok program sebagai pembiasaan dokumentasi kode.



6. Materi

6.1 Pengantar Algoritma dan Struktur Data

Dalam pemrograman, **algoritma** adalah urutan langkah logis dan sistematis untuk menyelesaikan suatu masalah. Algoritma yang baik harus:

- benar (correct)
- jelas (clear)
- efisien (efficient)

Sedangkan **struktur data** adalah cara mengorganisasi dan menyimpan data agar proses pengolahan data (misalnya pencarian, penambahan, penghapusan) menjadi lebih mudah dan lebih cepat.

Pada pertemuan pertama ini, fokus praktikum adalah membangun dasar kemampuan pengolahan data menggunakan Python dengan pendekatan **data berbasis file**.

6.2 Konsep ADT (Abstract Data Type)

ADT (Abstract Data Type) adalah konsep yang menjelaskan struktur data berdasarkan:

1. Data apa yang disimpan
2. Operasi apa yang dapat dilakukan terhadap data

Tanpa membahas bagaimana detail implementasinya di dalam program.

Contoh ADT yang umum:

- **ADT Koleksi (List/Dictionary/Set/Tuple)**

Operasi: tambah data, hapus data, akses data, cari data

- **ADT Stack (LIFO)**

Operasi: push, pop, peek

- **ADT Queue (FIFO)**

Operasi: enqueue, dequeue, front



Pada Modul 1 ini, mahasiswa akan menggunakan ADT koleksi sederhana untuk menampung data hasil pembacaan file, yaitu:

- **List**
- **Dictionary**

6.3 Konsep Pengolahan Data Berbasis File

Pada banyak aplikasi, data perlu disimpan secara permanen. Salah satu cara paling sederhana adalah menyimpan data ke dalam file teks.

Konsep penting:

- File digunakan untuk menyimpan data secara permanen
- Memori (RAM) digunakan untuk memproses data saat program berjalan

Alur pengolahan data berbasis file:

1. Data disimpan pada file (.txt)
2. Program membaca file (file ke memori)
3. Data dimasukkan ke struktur data (list/dictionary)
4. Data dimanipulasi (misalnya tambah atau ubah)
5. Data disimpan kembali ke file (memori ke file)

6.4 File Handling (File I/O) di Python

File Handling adalah proses membaca dan menulis file melalui program.

Python menyediakan fungsi `open()` untuk mengakses file.

Mode akses file

- "r" : membaca file (read)
- "w" : menulis file (write) dan **menimpa** isi file lama
- "a" : menambahkan data di akhir file (append)

Pada praktikum ini, format yang digunakan adalah:

```
main.py > ...  
1 with open("nama_file.txt", "r", encoding="utf-8") as file:  
2     data = file.read()
```

Kelebihan with open(...):

- file otomatis tertutup setelah selesai digunakan
- mengurangi risiko file “terbuka” dan menyebabkan error

6.5 Struktur Data Sederhana untuk Menampung Data File

Setelah file dibaca, data perlu disimpan ke struktur data agar mudah diproses.

Bentuk penampungan data yaitu

- **List**

List cocok untuk menyimpan data dalam bentuk urutan (sequence).

Contoh format data dalam list: ["J0403001", "Ani", 88]

- **Dictionary**

Dictionary cocok untuk menyimpan data berbentuk pasangan **key-value**, sehingga akses data lebih cepat. Contoh:

- key = NIM
- value = informasi mahasiswa

```
{  
    "j0403001": {"nama": "Ani", "nilai": 88},  
    "j0403002": {"nama": "Budi", "nilai": 75}  
}
```

6.6 Kesimpulan Materi Modul 1

Pada Modul Praktikum 1 ini, mahasiswa mempelajari dasar yang sangat penting untuk praktikum berikutnya dan proyek akhir, yaitu:

1. Konsep ADT sebagai dasar memahami struktur data
2. Pengolahan data berbasis file sebagai bentuk penyimpanan sederhana
3. File I/O Python (read/write)
4. Penyimpanan data file ke struktur data (list/dictionary) untuk diproses



7. Langkah Praktikum

7.1 Persiapan Folder dan File Praktikum

Buat folder kerja dengan nama: Praktikum ASD 1

Di dalam folder tersebut, buat 2 file berikut:

- praktikum1.py
- data_mahasiswa.txt

7.2 Membuat Dataset Awal

Keterangan format data:

- Kolom 1 = NIM
- Kolom 2 = Nama
- Kolom 3 = Nilai

Isi file data_mahasiswa.txt dengan data berikut:

J0403001,Alya,88

J0403002,Bima,75

J0403003,Citra,92

J0403004,Dimas,81

J0403005,Eka,69

J0403006,Fajar,85

J0403007,Gita,90

J0403008,Hana,77

J0403009,Indra,84

J0403010,Jihan,95



7.3 Latihan - Latihan

Latihan Dasar

Bagian ini menyajikan latihan secara bertahap. Mahasiswa memulai dari membaca file saja, kemudian parsing, menyimpan data ke List, mengenal Dictionary, hingga mengerjakan studi kasus pengolahan data mahasiswa.

Latihan Dasar 1A : Membaca Seluruh Isi File (read())

Tujuan: Mahasiswa memahami cara membaca isi file sekaligus menggunakan read().

Kode program:

```
# =====
# Praktikum 1: Konsep ADT dan File Handling
# Latihan Dasar 1A: Membaca seluruh isi file
# =====

# Membuka file dalam mode read ("r")
with open("data_mahasiswa.txt", "r", encoding="utf-8") as file:
    isi_file = file.read() # Membaca seluruh isi file sebagai string
# Menampilkan isi file ke layar
print("=== Isi File Data Mahasiswa ===")
print(isi_file)
```

Checkpoint (Validasi):

- Output menampilkan seluruh isi file.
- Mahasiswa memahami bahwa hasil read() bertipe string.

Latihan Dasar 1B : Membaca File Per Baris

Tujuan: Mahasiswa memahami cara membaca file baris demi baris dan menghilangkan newline dengan strip().

Kode program:

```
# =====
# Praktikum 1: Konsep ADT dan File Handling
# Latihan Dasar 1B: Membaca file per baris
# =====

with open("data_mahasiswa.txt", "r", encoding="utf-8") as file:
    for baris in file:
        baris = baris.strip() # Menghapus karakter newline
        print(baris)
```

Checkpoint (Validasi):

- Output menampilkan tiap baris data tanpa baris kosong tambahan.
- Mahasiswa memahami fungsi strip().



Latihan Dasar 1C : Analisis Perbedaan read() vs Per Baris

Tujuan: Mahasiswa melihat perbedaan bentuk data hasil read() (1 string) vs per baris (banyak string).

Kode program:

```
# =====
# Praktikum 1: Konsep ADT dan File Handling
# Latihan Dasar 1C: Analisis read() vs baca per baris
# =====

# -----
# Bagian A: Baca file dengan read()
# -----
with open("data_mahasiswa.txt", "r", encoding="utf-8") as file:
    isi_file = file.read() # seluruh isi file jadi 1 string besar

print("=== HASIL READ() ===")
print("Tipe data:", type(isi_file))
print("Jumlah karakter:", len(isi_file))
print("Jumlah baris:", isi_file.count("\n") + 1)

# -----
# Bagian B: Baca file per baris
# -----
jumlah_baris = 0
with open("data_mahasiswa.txt", "r", encoding="utf-8") as file:
    for baris in file:
        jumlah_baris = jumlah_baris + 1
        baris = baris.strip()
        print("Baris ke-", jumlah_baris)
        print("Isinya :", baris)
```

Checkpoint (Validasi):

- Mahasiswa dapat menjelaskan: read() menghasilkan 1 string besar.
- Membaca per baris menghasilkan banyak string (tiap baris).

Latihan Dasar 2 : Parsing Data (split(','))

Tujuan: Mahasiswa mampu memecah 1 baris menjadi tiga kolom: NIM, Nama, Nilai.

Kode program:

```
# =====
# Latihan Dasar 2: Parsing baris menjadi kolom (split)
# =====

with open("data_mahasiswa.txt", "r", encoding="utf-8") as file:
    for baris in file:
        baris = baris.strip()
        nim, nama, nilai = baris.split(",") # parsing 3 kolom
        print("NIM:", nim, "| Nama:", nama, "| Nilai:", nilai)
```

Checkpoint (Validasi):

- Output menampilkan kolom NIM, Nama, Nilai untuk setiap baris.
- Mahasiswa memahami bahwa split(',') menghasilkan list kolom.

Latihan Dasar 3 : Menyimpan Data ke List (List of Lists)

Tujuan: Mahasiswa mengenal struktur data List untuk menampung banyak record.

Kode program:

```
# =====
# Praktikum 1: Konsep ADT dan File Handling
# Latihan Dasar 3: Membaca file dan menyimpan ke struktur data List
# =====

data_list = [] # List untuk menampung data mahasiswa

with open("data_mahasiswa.txt", "r", encoding="utf-8") as file:
    for baris in file:
        baris = baris.strip()

        # Jika baris kosong, lewati
        if baris == "":
            continue

        # Pecah data berdasarkan koma
        nim, nama, nilai = baris.split(",")

        # Simpan sebagai list kecil: [nim, nama, nilai]
        data_list.append([nim, nama, int(nilai)])

print("=== Data Mahasiswa dalam List ===")
print(data_list)

print("=== Jumlah Record dalam List ===")
print("Jumlah record:", len(data_list))

print("=== Menampilkan Data Record Tertentu ===")
print("Contoh record pertama:", data_list[0])
```

Checkpoint (Validasi):

- data_list berisi sejumlah record (misal 10).
- Mahasiswa memahami List sebagai kumpulan record berurutan.



Latihan Dasar 4 : Menyimpan Data ke Dictionary (Key = NIM)

Tujuan: Mahasiswa mengenal Dictionary untuk akses data cepat menggunakan key (NIM).

Kode program:

```
# =====
# Praktikum 1: Konsep ADT dan File Handling
# Latihan Dasar 4A: Membaca file dan menyimpan ke struktur data Dictionary
# =====

data_dict = {} # Dictionary: key = NIM, value = data mahasiswa

with open("data_mahasiswa.txt", "r", encoding="utf-8") as file:
    for baris in file:
        baris = baris.strip()

        # Jika baris kosong, lewati
        if baris == "":
            continue

        nim, nama, nilai = baris.split(",")

        # Simpan data mahasiswa dengan key NIM
        data_dict[nim] = {
            "nama": nama,
            "nilai": int(nilai)
        }

print("=== Data Mahasiswa dalam Dictionary ===")
print(data_dict)
```



Latihan Studi Kasus

Bagian ini berisi latihan menggunakan studi kasus Data Mahasiswa. Setiap latihan dilakukan bertahap untuk membangun pemahaman mahasiswa tentang proses Read, Simpan di struktur data, Tampilkan, Cari, Update, Simpan (Write).

Latihan 1 : Load Data (Read File : Dictionary)

Tujuan: Mahasiswa mampu membaca file data_mahasiswa.txt dan menyimpannya ke struktur data dictionary.

Kode program:

```
# =====
# Praktikum 1: ADT & File Handling (Studi Kasus: Data Mahasiswa)
# Latihan 1: Load data dari file ke Dictionary
# =====

NAMA_FILE = "data_mahasiswa.txt"

def baca_data_mahasiswa(nama_file):
    """
    Membaca data mahasiswa dari file.
    Format per baris: NIM,NAMA,NILAI

    Output:
    - data_dict (dictionary)
      key    = NIM
      value = {"nama": NAMA, "nilai": NILAI(int)}
    """
    data_dict = {}

    with open(nama_file, "r", encoding="utf-8") as file:
        for baris in file:
            baris = baris.strip()

            # Lewati baris kosong
            if baris == "":
                continue

            parts = baris.split(",")
            if len(parts) != 3:
                continue

            nim, nama, nilai_str = parts

            try:
                nilai_int = int(nilai_str)
            except ValueError:
                continue

            data_dict[nim] = {"nama": nama, "nilai": nilai_int}

    return data_dict
```

```
# --- Program utama Latihan 1 ---
```

```
data_mahasiswa = baca_data_mahasiswa(NAMA_FILE)

print("=== Latihan 1 Selesai ===")
print("Jumlah data mahasiswa terbaca:", len(data_mahasiswa))
```

Checkpoint (Validasi):

- Program berhasil membaca file data_mahasiswa.txt.
- Jumlah data mahasiswa yang terbaca sesuai jumlah baris pada file (contoh: 10).

Latihan 2 . Tampilkan Semua Data

Tujuan: Mahasiswa mampu menampilkan seluruh data mahasiswa dalam format tabel yang rapi.

Kode program (lanjutan dari Latihan 1):

```
# =====
# Latihan 2: Tampilkan semua data mahasiswa
# =====

def tampilkan_semua_mahasiswa(data_dict):
    """
    Menampilkan semua data mahasiswa dalam format tabel.
    """
    if len(data_dict) == 0:
        print("Data mahasiswa kosong.")
        return
    '''Untuk tampilan yang rapi, atur f-string formatting.
    {nim:<10} artinya:
    tampilkan nim < = rata kiri dengan 10 = lebar kolom 10 karakter
    {nama:<12} artinya:
    tampilkan nama rata kiri. lebar kolom 12 karakter
    {nilai:>5} artinya:
    tampilkan nilai > = rata kanan, lebar kolom 5 karakter (biar angka sejajar)
    ...
    print("\n=== DAFTAR MAHASISWA ===")
    #Membuat header tabel
    print(f'{'NIM':<10} | {'Nama':<12} | {'Nilai':>5}')
    print("-" * 32) #buat garis header tabel

    #Pengulangan untuk mencetak data
    for nim in sorted(data_dict.keys()):
        nama = data_dict[nim]["nama"]
        nilai = data_dict[nim]["nilai"]
        print(f'{'nim':<10} | {'nama':<12} | {'nilai':>5}')
```

```
# --- Program utama Latihan 2 ---
tampilkan_semua_mahasiswa(data_mahasiswa)
```

Checkpoint (Validasi):

- Output menampilkan tabel berisi NIM, Nama, dan Nilai.
- Jumlah baris data yang tampil sesuai jumlah data mahasiswa.

Latihan 3 . Cari Mahasiswa Berdasarkan NIM (Input User)

Tujuan: Mahasiswa mampu mencari data mahasiswa berdasarkan NIM menggunakan dictionary.

Kode program (lanjutan dari Latihan 1):

```
# =====
# Latihan 3: Cari data mahasiswa berdasarkan NIM
# =====

def cari_mahasiswa(data_dict):
    """
    Mencari mahasiswa berdasarkan NIM (key dictionary).
    """
    nim_cari = input("Masukkan NIM yang ingin dicari (contoh: J0403001): ").strip()

    if nim_cari in data_dict:
        nama = data_dict[nim_cari]["nama"]
        nilai = data_dict[nim_cari]["nilai"]

        print("\n== Data Mahasiswa Ditemukan ==")
        print(f"NIM    : {nim_cari}")
        print(f>Nama   : {nama}")
        print(f"Nilai  : {nilai}")
    else:
        print("\nData tidak ditemukan. Pastikan NIM yang dimasukkan benar.")

# --- Program utama Latihan 3 ---
cari_mahasiswa(data_mahasiswa)
```

Checkpoint (Validasi):

- Jika input NIM valid (misal J0403001), program menampilkan data mahasiswa.
- Jika input NIM tidak ada (misal J0403099), program menampilkan pesan 'Data tidak ditemukan'.

Latihan 4 . Update Nilai Mahasiswa

Tujuan: Mahasiswa mampu memperbarui nilai mahasiswa berdasarkan NIM dengan validasi nilai 0–100.

Kode program (lanjutan dari Latihan 1):

```
# =====
# Latihan 4: Update nilai mahasiswa (ubah nilai)
# =====

def update_nilai_mahasiswa(data_dict):
    """
    Mengubah nilai mahasiswa berdasarkan NIM.
    Aturan:
    - NIM harus ada
    - Nilai baru harus 0-100
    """
    nim = input("Masukkan NIM yang ingin diupdate nilainya: ").strip()

    if nim not in data_dict:
        print("NIM tidak ditemukan. Update dibatalkan.")
```

```
        return

    try:
        nilai_baru = int(input("Masukkan nilai baru (0-100): ").strip())
    except ValueError:
        print("Nilai harus berupa angka. Update dibatalkan.")
        return

    if nilai_baru < 0 or nilai_baru > 100:
        print("Nilai harus antara 0 sampai 100. Update dibatalkan.")
        return

    nilai_lama = data_dict[nim]["nilai"]
    data_dict[nim]["nilai"] = nilai_baru

    print(f"Update berhasil. Nilai {nim} berubah dari {nilai_lama} menjadi {nilai_baru}.")

# --- Program utama Latihan 4 ---
update_nilai_mahasiswa(data_mahasiswa)

# Menampilkan ulang tabel untuk memastikan data berubah
tampilkan_semua_mahasiswa(data_mahasiswa)
```

Checkpoint (Validasi):

- Jika NIM valid, nilai mahasiswa berubah sesuai input.
- Jika NIM tidak valid, program menampilkan pesan gagal update.
- Jika nilai di luar 0–100, update dibatalkan.

Latihan 5. Simpan Perubahan ke File (Write File)

Tujuan: Mahasiswa mampu menyimpan data mahasiswa yang sudah diperbarui ke file `data_mahasiswa.txt`.

Kode program (lanjutan dari Latihan 1):

```
# =====
# Latihan 5: Simpan perubahan data mahasiswa ke file (WRITE)
# =====

def simpan_data_mahasiswa(nama_file, data_dict):
    """
    Menyimpan data mahasiswa dari dictionary ke file.
    Format per baris: NIM,NAMA,NILAI
    """
    with open(nama_file, "w", encoding="utf-8") as file:
        for nim in sorted(data_dict.keys()):
            nama = data_dict[nim]["nama"]
            nilai = data_dict[nim]["nilai"]
            file.write(f"{nim},{nama},{nilai}\n")

# --- Program utama Latihan 5 ---
simpan_data_mahasiswa(NAMA_FILE, data_mahasiswa)
print("Data berhasil disimpan ke file:", NAMA_FILE)
```



Checkpoint (Validasi):

- File data_mahasiswa.txt berubah sesuai hasil update nilai pada Latihan 4.
- Jumlah baris tetap sama dan format tetap NIM,NAMA,NILAI.

Latihan 6 . Gabungan Seluruh Latihan Berbasis Menu

Latihan Final ini menggabungkan seluruh latihan sebelumnya menjadi satu program utuh berbasis menu. Mahasiswa diminta menjalankan program, melakukan pencarian NIM, mengubah nilai, dan menyimpan kembali data ke file.

Kode Program (praktikum1.py):

```
# =====
# Praktikum 1: ADT & File Handling (Studi Kasus: Data Mahasiswa)
# Latihan Final: Menu Interaktif (Gabungan Latihan 1-5)
# =====

NAMA_FILE = "data_mahasiswa.txt"

# -----
# 1) Read: Baca data dari file
# -----
def baca_data_mahasiswa(nama_file):
    """
    Membaca data mahasiswa dari file.
    Format per baris: NIM,NAMA,NILAI

    Output:
    - data_dict (dictionary)
      key    = NIM
      value = {"nama": NAMA, "nilai": NILAI(int)}
    """
    data_dict = {}

    # Jika file belum ada, kembalikan dictionary kosong (aman)
    try:
        with open(nama_file, "r", encoding="utf-8") as file:
            for baris in file:
                baris = baris.strip()

                # Lewati baris kosong
                if baris == "":
                    continue

                parts = baris.split(",")
                if len(parts) != 3:
                    continue

                nim, nama, nilai_str = parts

                try:
                    nilai_int = int(nilai_str)
                except ValueError:
                    continue
```




```
data_dict[nim] = {"nama": nama, "nilai": nilai_int}

except FileNotFoundError:
    # File tidak ditemukan → data kosong
    pass

return data_dict

# -----
# 2) Tampilkan semua data
# -----
def tampilkan_semua_mahasiswa(data_dict):
    """
    Menampilkan semua data mahasiswa dalam format tabel.
    """
    if len(data_dict) == 0:
        print("Data mahasiswa kosong.")
        return

    print("\n=== DAFTAR MAHASISWA ===")
    print(f"{'NIM':<10} | {'Nama':<12} | {'Nilai':>5}")
    print("-" * 32)

    for nim in sorted(data_dict.keys()):
        nama = data_dict[nim]["nama"]
        nilai = data_dict[nim]["nilai"]
        print(f"{nim:<10} | {nama:<12} | {nilai:>5}")

# -----
# 3) Cari mahasiswa berdasarkan NIM
# -----
def cari_mahasiswa(data_dict):
    """
    Mencari mahasiswa berdasarkan NIM (key dictionary).
    """
    nim_cari = input("Masukkan NIM yang ingin dicari (contoh: J0403001): ").strip()

    if nim_cari in data_dict:
        nama = data_dict[nim_cari]["nama"]
        nilai = data_dict[nim_cari]["nilai"]

        print("\n=== Data Mahasiswa Ditemukan ===")
        print(f"NIM    : {nim_cari}")
        print(f>Nama   : {nama}")
        print(f"Nilai  : {nilai}")
    else:
        print("\nData tidak ditemukan. Pastikan NIM yang dimasukkan benar.")

# -----
# 4) Update nilai mahasiswa
# -----
def update_nilai_mahasiswa(data_dict):
    """
    Mengubah nilai mahasiswa berdasarkan NIM.
    Aturan:
    - NIM harus ada

```



```
- Nilai baru harus 0-100
"""
nim = input("Masukkan NIM yang ingin diupdate nilainya: ").strip()

if nim not in data_dict:
    print("NIM tidak ditemukan. Update dibatalkan.")
    return

try:
    nilai_baru = int(input("Masukkan nilai baru (0-100): ").strip())
except ValueError:
    print("Nilai harus berupa angka. Update dibatalkan.")
    return

if nilai_baru < 0 or nilai_baru > 100:
    print("Nilai harus antara 0 sampai 100. Update dibatalkan.")
    return

nilai_lama = data_dict[nim]["nilai"]
data_dict[nim]["nilai"] = nilai_baru

print(f"Update berhasil. Nilai {nim} berubah dari {nilai_lama} menjadi {nilai_baru}.")

# -----
# 5) Write: Simpan data ke file
# -----
def simpan_data_mahasiswa(nama_file, data_dict):
    """
    Menyimpan data mahasiswa dari dictionary ke file.
    Format per baris: NIM,NAMA,NILAI
    """
    with open(nama_file, "w", encoding="utf-8") as file:
        for nim in sorted(data_dict.keys()):
            nama = data_dict[nim]["nama"]
            nilai = data_dict[nim]["nilai"]
            file.write(f"{nim},{nama},{nilai}\n")

# =====
# Program Utama: Menu Interaktif
# =====
def main():
    # Load data saat program mulai
    data_mahasiswa = baca_data_mahasiswa(NAMA_FILE)

    while True:
        print("\n=== MENU DATA MAHASISWA ===")
        print("1. Tampilkan semua data")
        print("2. Cari data berdasarkan NIM")
        print("3. Update nilai mahasiswa")
        print("4. Simpan data ke file")
        print("0. Keluar")

        pilihan = input("Pilih menu: ").strip()

        if pilihan == "1":
            tampilkan_semua_mahasiswa(data_mahasiswa)
```

```
elif pilihan == "2":
    cari_mahasiswa(data_mahasiswa)

elif pilihan == "3":
    update_nilai_mahasiswa(data_mahasiswa)

elif pilihan == "4":
    simpan_data_mahasiswa(NAMA_FILE, data_mahasiswa)
    print("Data berhasil disimpan.")

elif pilihan == "0":
    print("Program selesai.")
    break

else:
    print("Pilihan tidak valid. Silakan coba lagi.")

if __name__ == "__main__":
    main()
```

Checkpoint (Validasi):

- Menu tampil dan dapat menerima input pilihan pengguna.
- Menu 2 berhasil mencari mahasiswa berdasarkan NIM yang valid.
- Menu 3 berhasil mengubah nilai mahasiswa dengan validasi 0–100.
- Menu 4 menyimpan perubahan ke file data_mahasiswa.txt.
- Program dapat keluar dengan menu 0 tanpa error.

8. Tugas Hands On dan Pengumpulan Tugas

8.1 Instruksi Tugas Hands-on (Stok Barang Kantin)

Tujuan: Mahasiswa mampu membangun program sederhana berbasis file yang dapat membaca, memproses, dan menyimpan data stok barang menggunakan struktur data dictionary.

A. Dataset Tugas

Gunakan file stok_barang.txt dengan format: KodeBarang,NamaBarang,Stok

Contoh:

BRG001,Roti,20

BRG002,AirMineral,15

BRG003,TehBotol,10

B. Ketentuan Program

- ☐ Data dibaca dari file stok_barang.txt saat program dimulai
- ☐ Data disimpan di memori menggunakan dictionary (key = KodeBarang)
- ☐ Program memiliki menu interaktif (loop) sampai user memilih keluar

- ☐ Program dapat menyimpan perubahan stok kembali ke file
- ☐ Setiap bagian kode wajib diberi komentar sebagai dokumentasi

C. Menu Minimal yang Harus Ada

- 1) Menu 1 : Tampilkan semua barang.
Menampilkan seluruh data stok barang (kode, nama, stok).
- 2) Menu 2 : Cari barang berdasarkan kode
Input kode barang dan tampilkan data jika ditemukan. Jika tidak ditemukan, tampilkan pesan 'Barang tidak ditemukan'.
- 3) Menu 3 : Tambah barang baru
Input kode, nama, stok awal. Jika kode sudah ada, tampilkan pesan 'Kode sudah digunakan'.
- 4) Menu 4 : Update stok barang
Input kode barang, pilih tambah stok atau kurangi stok. Stok tidak boleh negatif.
- 5) Menu 5 : Simpan ke file
Simpan seluruh data terbaru ke stok_barang.txt.
- 6) Menu 0 : Keluar
Keluar dari program dan tampilkan pesan 'Program selesai.'

8.2 Output Praktikum

Pada akhir Praktikum Modul 1, mahasiswa menghasilkan output berupa program Python berbasis file yang mampu:

- Membaca data dari file teks (.txt)
- Menyimpan data ke struktur data di memori (List/Dictionary)
- Menampilkan data dan melakukan manipulasi sederhana (tambah/ubah)
- Menyimpan kembali data ke file

Catatan:

- Latihan menggunakan studi kasus Data Mahasiswa
- Tugas Hands-on menggunakan studi kasus Stok Barang Kantin (berbeda dari latihan)

Mahasiswa wajib mengumpulkan file berikut:

1. praktikum1.py
2. tugas1.py
3. data_mahasiswa.txt (hasil latihan terarah)
4. stok_barang.txt (dataset untuk tugas hands-on)



Semua file dikumpulkan dalam 1 folder dengan format: **Modul1_NIM_Nama**

Contoh: Modul1_j0403001_Ani

8.3 Kriteria Keberhasilan Tugas Praktikum Modul 1

Praktikum Modul 1 dinyatakan memenuhi apabila:

- ☐ Program berhasil membaca file latihan (data_mahasiswa.txt)
- ☐ Program berhasil membaca file tugas (stok_barang.txt)
- ☐ Data berhasil disimpan ke struktur data (List/Dictionary)
- ☐ Program dapat menambah atau mengubah minimal 1 data
- ☐ Data dapat disimpan kembali ke file
- ☐ Kode program memiliki komentar sebagai dokumentasi dasar



Lampiran . Template Skeleton Program (tugas1.py)

Mahasiswa diminta melengkapi bagian TODO sesuai instruksi tugas hands-on.

```
# =====
# TUGAS HANDS-ON MODUL 1
# Studi Kasus: Sistem Stok Barang Kantin (Berbasis File .txt)
#
# Nama :
# NIM :
# Kelas :
# =====

# -----
# Konstanta nama file
# -----
NAMA_FILE = "stok_barang.txt"

# -----
# Fungsi: Membaca data dari file
# -----
def baca_stok(nama_file):
    """
    Membaca data stok dari file teks.
    Format per baris: KodeBarang>NamaBarang,Stok

    Output:
    - stok_dict (dictionary)
      key    = kode_barang
      value = {"nama": nama_barang, "stok": stok_int}
    """
    stok_dict = {}

    # TODO: Buka file dan baca seluruh baris
    # Hint: with open(nama_file, "r", encoding="utf-8") as f:

    # TODO: Untuk setiap baris:
    # - gunakan strip() untuk menghilangkan \n
    # - split(",") untuk memisahkan kolom
    # - simpan ke dictionary

    return stok_dict

# -----
# Fungsi: Menyimpan data ke file
# -----
def simpan_stok(nama_file, stok_dict):
    """
    Menyimpan seluruh data stok ke file teks.
    Format per baris: KodeBarang>NamaBarang,Stok
    """
    # TODO: Tulis ulang seluruh isi file berdasarkan stok_dict
    # Hint: with open(nama_file, "w", encoding="utf-8") as f:
    pass
```



```
# -----
# Fungsi: Menampilkan semua data
# -----
def tampilkan_semua(stok_dict):
    """
    Menampilkan semua barang di stok_dict.
    """
    # TODO: Jika kosong, tampilkan pesan stok kosong
    # TODO: Tampilkan dengan format rapi: kode | nama | stok
    pass

# -----
# Fungsi: Cari barang berdasarkan kode
# -----
def cari_barang(stok_dict):
    """
    Mencari barang berdasarkan kode barang.
    """
    kode = input("Masukkan kode barang: ").strip()

    # TODO: Cek apakah kode ada di dictionary
    # Jika ada: tampilkan detail barang
    # Jika tidak ada: tampilkan 'Barang tidak ditemukan'
    pass

# -----
# Fungsi: Tambah barang baru
# -----
def tambah_barang(stok_dict):
    """
    Menambah barang baru ke stok_dict.
    """
    kode = input("Masukkan kode barang baru: ").strip()
    nama = input("Masukkan nama barang: ").strip()

    # TODO: Validasi kode tidak boleh duplikat
    # Jika sudah ada: tampilkan 'Kode sudah digunakan' dan return

    # TODO: Input stok awal (integer)
    # Hint: stok_awal = int(input(...))

    # TODO: Simpan ke dictionary
    pass

# -----
# Fungsi: Update stok barang
# -----
def update_stok(stok_dict):
    """
    Mengubah stok barang (tambah atau kurangi).
    Stok tidak boleh menjadi negatif.
    """
    kode = input("Masukkan kode barang yang ingin diupdate: ").strip()
```



```
# TODO: Cek apakah kode ada di dictionary
# Jika tidak ada: tampilkan pesan dan return

print("Pilih jenis update:")
print("1. Tambah stok")
print("2. Kurangi stok")

pilihan = input("Masukkan pilihan (1/2): ").strip()

# TODO: Input jumlah perubahan stok
# Hint: jumlah = int(input("Masukkan jumlah: "))

# TODO:
# - Jika pilihan 1: stok = stok + jumlah
# - Jika pilihan 2: stok = stok - jumlah
# - Jika hasil < 0: batalkan dan tampilkan error

pass

# -----
# Program Utama
# -----
def main():
    # Membaca data dari file saat program mulai
    stok_barang = baca_stok(NAMA_FILE)

    while True:
        print("\n=== MENU STOK KANTIN ===")
        print("1. Tampilkan semua barang")
        print("2. Cari barang berdasarkan kode")
        print("3. Tambah barang baru")
        print("4. Update stok barang")
        print("5. Simpan ke file")
        print("0. Keluar")

        pilihan = input("Pilih menu: ").strip()

        if pilihan == "1":
            tampilkan_semua(stok_barang)

        elif pilihan == "2":
            cari_barang(stok_barang)

        elif pilihan == "3":
            tambah_barang(stok_barang)

        elif pilihan == "4":
            update_stok(stok_barang)

        elif pilihan == "5":
            simpan_stok(NAMA_FILE, stok_barang)
            print("Data berhasil disimpan.")

        elif pilihan == "0":
            print("Program selesai.")
            break
```




```
else:
    print("Pilihan tidak valid. Silakan coba lagi.")

# Menjalankan program utama
if __name__ == "__main__":
    main()
```